



AUTONOMOUS VEHICLE MANAGEMENT SYSTEM

Assignment: Data Structures and Algorithms
COMP5008

Contents

Table of Figures	2
1. Introduction	3
2. Class and Method Descriptions	3
DSAGraph Class -	3
VehicleHashTable Class -	4
Vehicle Class -	6
Sorting Class -	6
Flowchart –	10
3. Test Cases	11
Graph Test –	11
VehicleHashTable Test -	11
Vehicle Test -	12
Sorting Test -	12
4. Challenges and Solutions	13
Avoiding Built-in Functions of Python	13
Efficient Sorting Implementation	13
5. Efficiency Analysis and Potential Improvements	14
Efficiency Analysis:	14
Potential Improvements:	14
6. Conclusion	15
7. Reference	15

Table of Figures

Figure 1 – def add_node	3
Figure 2 – def add_edge	3
Figure 3 - def is_path	4
Figure 4 – def display_graph	4
Figure 5 -def insert	5
Figure 6 - def search	5
Figure 7 - def delete	5
Figure 8 - def display	5
Figure 9 - vehicle class def	6
Figure 10 - def heapsort	7
Figure 12 - def get_node_at_index	7
Figure 13 - def swap_nodes	7
Figure 14 - def heapify	8
Figure 15 - def quicksort	8
Figure 16 - def partition	8
Figure 17 - Flowchart	11
Figure 18 - Results from graph test	11
Figure 19 - Results from vehicle_hash_table test	12
Figure 20 - Results from vehicle test	12
Figure 21 - Results from sorting test	13

1. Introduction

The aim of this report is to explain the implementation of a **data structure-driven vehicle management system** for an autonomous vehicle fleet. The program will efficiently allow the user to add locations and destinations and create roads between them. Similarly, the user will be able to add vehicles with unique vehicle ID along with other information related to the vehicle. This program handles other operations like pathfinding between locations, and sorting vehicles based on distance to destination or battery level. The system is built using linked lists, hash tables, sorting algorithms such as heapsort, quicksort, and a graph to manage locations and vehicle routes without using Python's built-in list or dictionary functions.

2. Class and Method Descriptions

DSAGraph Class -

Purpose of the DSAGraph is to manage locations and the roads between the two locations using nodes in a graph. This is done using multiple key methods such as, “add_node” to add locations to the graph, “add_edge” to create an edge which will represent a road between two nodes which represents locations, “is_path” to check existence of a path between two locations using Depth First Search and “display_graph” to print all the nodes and the adjacent locations ("Depth first search or DFS for a graph," 2023).

```
# Add a new node to the graph if it doesn't already exist
def add_node(self, value):
    if not self.nodes.contains(DSAGraphNode(value)): # Check if node exists before adding
        self.nodes.add(DSAGraphNode(value))
```

Figure 1 – def add_node

```
# Add an edge between two nodes in the graph
def add_edge(self, src_value, dest_value):
    src_node = self.find_node(src_value) # Find source node
    dest_node = self.find_node(dest_value) # Find destination node
    if src_node and dest_node: # If both nodes exist, add them as adjacent to each other
        src_node.add_adjacent(dest_node)
        dest_node.add_adjacent(src_node)
```

Figure 2 – def add_edge

```

# Check if a path exists between two nodes using depth-first search
def is_path(self, source, destination):
    source_node = self.find_node(source)
    destination_node = self.find_node(destination)

    if source == destination:
        print("Source and destination is the same.")
        return True

    if source_node is None or destination_node is None:
        print(f"Either {source} or {destination} does not exist in the graph.")
        return False

    # Call the DFS function to check for a path
    return self.dfs(source, destination)

```

Figure 3 - def is_path

```

# Display the graph with each node, its adjacent nodes, and the distances
def display_graph(self):
    # Traverse through all the nodes in the graph
    current = self.nodes.get_head() # Get the head node of the graph's custom linked list
    while current is not None:
        node = current.value # Get the current node
        print(f"{node.value}: ", end="") # Print the node's value

        # Get the adjacent nodes and print them along with distances
        neighbor = node.get_adjacent() # Get the adjacent nodes' head from the custom linked list
        while neighbor is not None:
            print(f"{neighbor.value.value} (Distance: {neighbor.distance}) ", end="") # Print the adjacent node's value and distance
            neighbor = neighbor.next # Move to the next adjacent node

        print()
        current = current.next # Move to the next node in the graph's linked list

```

Figure 4 – def display_graph

Similarly, other functions such as retrieving neighbours were implemented to find the closest nodes of a selected node and display the results using the main.py. Also the display function prints all the nodes as well as the distances between the nodes representing locations and the distances between the two locations allowing the user to see the full idea of the given locations and the distances.

VehicleHashTable Class -

Purpose of the vehicle has table class is to stores vehicle information related data in a custom hash table where each bucket is a custom linked list ("Python linked list," 2023). The key methods used in this class are, “insert” to add vehicles to hash table using hash functions, “search” to search for vehicles by its vehicle ID, “delete” to remove a vehicle by its vehicle ID and finally, “display” to display all the stored vehicles in the custom hash table ("Implementation of hash table in Python using separate chaining," 2023).

```

# Insert a vehicle into the hash table
def insert(self, vehicle):
    if (self.count / self.size) > 0.75: # Resize when load factor exceeds 0.75
        self._resize()

    index = self._custom_hash(vehicle.vehicle_id)
    bucket = self.buckets.find(index)

    if bucket is None: # If no bucket exists at that index, create one
        new_bucket = VehicleLinkedList() # Create a new linked list bucket
        new_bucket.add(vehicle) # Add the vehicle to the bucket
        self.buckets.add(index, new_bucket) # Add the new bucket to the hash table
    else:
        bucket.add(vehicle) # Add the vehicle to the existing bucket

    self.count += 1 # Increment the vehicle count

```

Figure 5 - def insert

```

# Search for a vehicle in the hash table
def search(self, vehicle_id):
    index = self._custom_hash(vehicle_id)
    bucket = self.buckets.find(index)
    if bucket is not None:
        return bucket.search(vehicle_id)
    return None

```

Figure 6 - def search

```

# Delete a vehicle from the hash table
def delete(self, vehicle_id):
    index = self._custom_hash(vehicle_id)
    bucket = self.buckets.find(index)
    if bucket is not None:
        bucket.remove(vehicle_id)
        self.count -= 1 # Decrement the vehicle count

```

Figure 7 - def delete

```

# Display all vehicles in the hash table
def display(self):
    self.buckets.display() # Display all the buckets and their vehicles

```

Figure 8 - def display

However, the hash tables which are used in the code are custom built hash tables as using the inbuilt hash table was not allowed. The size of the table is resizing using load factor when the size of the table reaches 0.75. The resizing is done using prime numbers as practiced on the practicals.

Vehicle Class -

Purpose of the vehicle class is to represents the details of a vehicle, including its “vehicle_id”, “location”, “destination”, “distance_to_dest”, and “battery_level”. These are achieved by using “set_location”, “set_destination”, “set_distance_to_dest”, and “set_battery_level” to allow setting up the vehicle attributes as well as “get_location”, “get_destination”, “get_distance_to_dest”, and “get_battery_level” to provide access to the attributes as key methods.

```
def set_location(self, location):
    self.location = location # Set the current location

def set_destination(self, destination):
    self.destination = destination # Set the destination

def set_distance_to_dest(self, distance):
    self.distance_to_dest = distance # Set the distance to the destination

def set_battery_level(self, level):
    self.battery_level = level # Set the battery level

def get_location(self):
    return self.location # Get the current location

def get_destination(self):
    return self.destination # Get the destination

def get_distance_to_dest(self):
    return self.distance_to_dest # Get the distance to the destination

def get_battery_level(self):
    return self.battery_level # Get the battery level
```

Figure 9 - vehicle class def

Sorting Class -

Purpose of the sorting class is to implement two sorting algorithms, which are Heapsort and Quicksort to sort vehicles based on different criteria such as “distance_to_dest” and “battery_level” ("Python program for QuickSort," 2023). Both algorithms are implemented directly on a linked list, avoiding Python's built-in list functions.

Multiple methods are used in sorting as listed below,

Heap sort(linked_list) function to sort vehicles by their distance to the destination using the heapify operation ("Python program for heap sort," 2024).

```

# Perform heapsort on the linked list without converting to an array
def heapsort(self):
    n = self.length()
    # Build the max heap
    for i in range(n // 2 - 1, -1, -1):
        self.heapify(n, i)
    # Extract elements from heap one by one
    for i in range(n - 1, 0, -1):
        self.swap(self.get_node_at_index(0), self.get_node_at_index(i))
        self.heapify(i, 0)

```

Figure 10 - def heapsort

This builds a max heap before swapping the largest element with the last element, reducing the heap size. This process continues until all elements are sorted. This leverages the “get_node_at_index” function to retrieve nodes from the linked list and “swap_nodes” to exchange data between nodes.

```

# Traverse the linked list to get the node at the specified index
def get_node_at_index(self, index):
    current = self.head
    for _ in range(index):
        if current is None:
            return None
        current = current.next
    return current

```

Figure 11 - def get_node_at_index

```

# Swap the vehicle objects of two linked list nodes
def swap(self, node1, node2):
    node1.vehicle, node2.vehicle = node2.vehicle, node1.vehicle

```

Figure 12 - def swap_nodes

Heapify (linked_list, n, i) function supports heapsort by maintaining the heap property while ensuring that the node at index “i” is larger than its children. This recursively apply to the subtree if necessary ("Heap queue (or heapq) in Python," 2023).


```

# Heapify a subtree rooted at index i in the linked list
def heapify(self, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    # Get the node at index 'largest', 'left', and 'right'
    largest_node = self.get_node_at_index(largest)
    left_node = self.get_node_at_index(left)
    right_node = self.get_node_at_index(right)

    # Check if left child exists and is larger than the root
    if left < n and left_node and left_node.vehicle.distance_to_dest > largest_node.vehicle.distance_to_dest:
        largest = left

    # Reassign 'largest_node' in case the largest has changed
    largest_node = self.get_node_at_index(largest)
    # Check if right child exists and is larger than the largest so far
    if right < n and right_node and right_node.vehicle.distance_to_dest > largest_node.vehicle.distance_to_dest:
        largest = right

    # Swap and heapify the subtree if needed
    if largest != i:
        # Swap the root and the largest node
        self.swap(self.get_node_at_index(i), self.get_node_at_index(largest))
        # Recursively heapify the affected subtree
        self.heapify(n, largest)

```

Figure 13 - def heapify

quicksort (linked_list, low, high) function uses quick sort algorithm to sort vehicles by battery level.

```

# Quicksort the linked list by battery level
def quicksort(self, low, high):
    if low < high:
        pi = self.partition(low, high)
        self.quicksort(low, pi - 1)
        self.quicksort(pi + 1, high)

```

Figure 14 - def quicksort

partition (linked_list, low, high) function supports quicksort by organizing the elements relative to the pivot node. This is to move all elements with a battery level equal or greater than to the pivot to one side of the pivot and the rest to the other side.

```

# Partition the linked list for quicksort, using the node at index 'high' as the pivot
def partition(self, low, high):
    pivot = self.get_node_at_index(high).vehicle.battery_level
    i = low - 1
    for j in range(low, high):
        if self.get_node_at_index(j).vehicle.battery_level >= pivot:
            i += 1
            self.swap(self.get_node_at_index(i), self.get_node_at_index(j))
    self.swap(self.get_node_at_index(i + 1), self.get_node_at_index(high))
    return i + 1

```

Figure 15 - def partition

Supporting Functions are “`get_node_at_index(linked_list, index)`” which retrieves the node at a specific index in the linked list. This is necessary for both sorting algorithms since random access is not available in a linked list. Similarly, “`swap_nodes(node1, node2)`” function swaps the vehicle data between two nodes which is essential for the implementation of heapsort and quicksort. This is mandatory as data needs to be swapped without using the python's in built list function.

Flowchart –

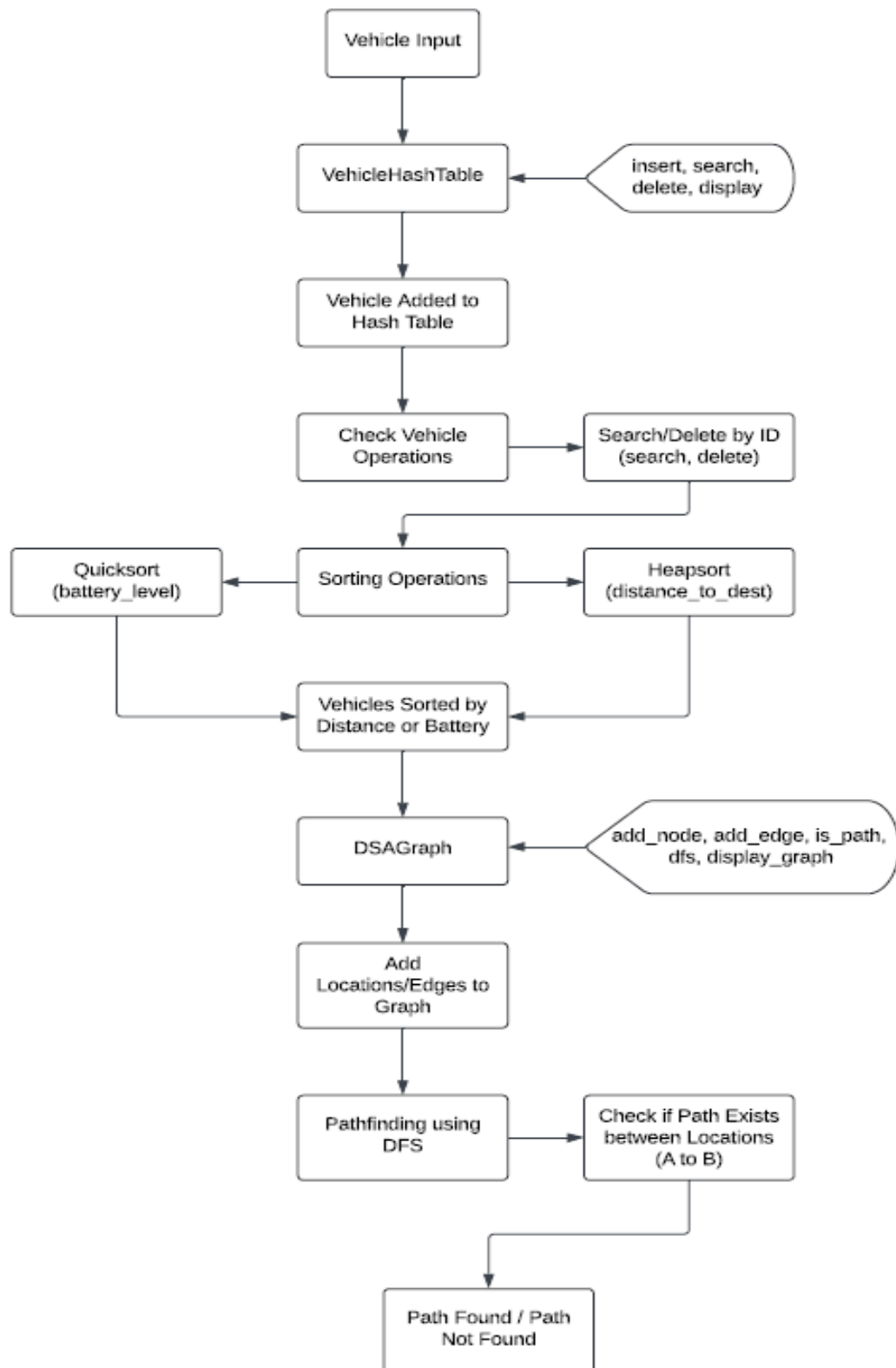


Figure 16 - Flowchart

3. Test Cases

Graph Test –

Graph.py is tested by giving it test inputs to add locations as A, B, and C with roads in between the locations. As the result of the test code the graph should display the nodes and paths while showing a path between the A and C.

```
PS C:\Users\jayat\OneDrive\Desktop\Curtin\Semester 2\COMP5008 - Data Structures and Algorithms\Assignment> & C:/Users/jayat/OneDrive/Desktop/Curtin\Semester 2\COMP5008 - Data Structures and Algorithms\Assignment> .py"
A: B (Distance: 20)
B: A (Distance: 20) C (Distance: 50)
C: B (Distance: 50)

Check is_path
Either A or D does not exist in the graph.

Check is_path when src=dest
Source and destination is the same.

Check is_path when source or destination is not the same
Either A or F does not exist in the graph.

No Error? Okay still working :)
PS C:\Users\jayat\OneDrive\Desktop\Curtin\Semester 2\COMP5008 - Data Structures and Algorithms\Assignment>
```

Figure 17 - Results from graph test

Also, the test code displays the existence of the locations with the given distances between the locations and assures that the destination and the starting location are not the same by giving an error mentioning that “Source and destination is the same”.

By trying to check paths between a location and a non-existing location shows an error mentioning that either one of the locations given does not exist.

VehicleHashTable Test -

Vehicle_hash_table.py is tested by Inserting multiple vehicles by giving ID as BMW, Mercedes, and Audi with various distances and battery levels. The vehicles are explorable and by deleting the vehicles that are removed from the hash table can be identified as the expected outcomes.

```

PS C:\Users\jayat\OneDrive\Desktop\Curtin\Semester 2\COMP5008 - Data Structures and Algorithms\Assignment> & C:/Users/jayat/AppData/Local/Programs/Python/Python39-6/Scripts/python.exe sh_table_test.py
Searching for Vehicle BMW...

Found Vehicle: ID=BMW, Location=A, Destination=B, Distance=15.0, Battery=80%

Displaying all vehicles in the hash table:

Bucket 0:
Vehicle ID BMW, Location: A, Destination: B, Distance: 15.0, Battery: 80%
Bucket 8:
Vehicle ID Mercedes, Location: A, Destination: B, Distance: 13.0, Battery: 50%
Bucket 7:
Vehicle ID Audi, Location: A, Destination: B, Distance: 11.0, Battery: 70%
PS C:\Users\jayat\OneDrive\Desktop\Curtin\Semester 2\COMP5008 - Data Structures and Algorithms\Assignment>

```

Figure 18 - Results from vehicle_hash_table test

Vehicle Test -

Vehicle.py is tested by giving inputs to create and manipulate vehicle instances such as Set the vehicle's location, destination, distance, and battery level using "set_location()", "set_destination()", "set_distance_to_dest()", and "set_battery_level()". These set values can be retrieved by using "get_location()", "get_destination()", "get_distance_to_dest()", and "get_battery_level()". As outcome the "get_" methods should return the correct values that were set using set_ methods.

```

PS C:\Users\jayat\OneDrive\Desktop\Curtin\Semester 2\COMP5008 - Data Structures and Algorithms\Assignment> & C:/Users/jayat/AppData/Local/Programs/Python/Python39-6/Scripts/python.exe st.py
Testing 1,2,3

Battery level is: 90
Destination is: D
Distance to destination is: 20.0
Location is: C
No error? Then it still works ;)

PS C:\Users\jayat\OneDrive\Desktop\Curtin\Semester 2\COMP5008 - Data Structures and Algorithms\Assignment>

```

Figure 19 - Results from vehicle test

Sorting Test -

Sorting.py is tested by creating vehicles with varying "distance_to_dest" and "battery_level" to have an outcome of vehicles to be sorted by "distance_to_dest" using heapsort and to be sorted by "battery_level" using quicksort.

```

PS C:\Users\jayat\OneDrive\Desktop\Curtin\Semester 2\COMP5008 - Data Structures and Algorithms\Assignment> & C:/Users/jay
st.py"
    Before sorting by distance to destination:

Vehicle ID V1, Location: A, Destination: B, Distance: 15, Battery: 80%
Vehicle ID V2, Location: C, Destination: D, Distance: 10, Battery: 90%
Vehicle ID V3, Location: E, Destination: F, Distance: 25, Battery: 60%

    After sorting by distance to destination:

Vehicle ID V2, Location: C, Destination: D, Distance: 10, Battery: 90%
Vehicle ID V1, Location: A, Destination: B, Distance: 15, Battery: 80%
Vehicle ID V3, Location: E, Destination: F, Distance: 25, Battery: 60%

    After sorting by battery level:

Vehicle ID V2, Location: C, Destination: D, Distance: 10, Battery: 90%
Vehicle ID V1, Location: A, Destination: B, Distance: 15, Battery: 80%
Vehicle ID V3, Location: E, Destination: F, Distance: 25, Battery: 60%
PS C:\Users\jayat\OneDrive\Desktop\Curtin\Semester 2\COMP5008 - Data Structures and Algorithms\Assignment>

```

Figure 20 - Results from sorting test

4. Challenges and Solutions

Avoiding Built-in Functions of Python

Lists function of Python's built-in and other built-in dictionary functions were not allowed to use. However, the use of custom created linked list to use for hash table buckets and graph adjacency lists were implemented as a solution. Similarly, the hash function was useful yet, using the built-in function wasn't allowed. Hence the use of a custom hash function was implemented.

Efficient Sorting Implementation

Heapsort and quicksort implementation directly on linked lists was complicated as of the absence of random access. Yet, by developing helper functions to traverse the linked list and swap nodes as required by the sorting algorithms.

5. Efficiency Analysis and Potential Improvements

Efficiency Analysis:

Graph Operations:

Depth First Search Pathfinding: The time complexity is " $O(V + E)$ ", where E is the number of edges representing roads while V is the number of vertices representing locations.

Hash Table:

Time Complexity - Insertion, deletion, and search operations are " $O(1)$ " on average but degrade to " $O(n)$ " in the worst-case scenario with collisions.

Space Complexity - The hash table uses " $O(n)$ " space, where n is the number of vehicles.

Sorting Algorithms:

Heapsort - Time complexity is " $O(n \log n)$ ".

Quicksort - Worst-case time complexity is " $O(n^2)$ ", but average-case is " $O(n \log n)$ ".

Potential Improvements:

- In the event of using significantly larger amount of data, implementing dynamic resizing is ideal instead of using fixed hash table size, so the table size will increase or decrease based on the load factors.
- When compared to a greater number of vehicles, implementing a more advanced collision resolution strategy in the hash table, such as quadratic probing or double hashing can further improve performance and reduce clustering.
- For a more detailed and user-friendly interface, a graphical user interface can be implemented. This will be helpful for the user to get a better understanding of the paths and the graph more easily.

6. Conclusion

The use of linked list such as hash tables and graph traversal can be successfully demonstrated by using the autonomous vehicle management system. Discovering solutions for challenges including, managing and sorting linked list data to avoid built in functions and dictionaries of python. However, the future improvements may enhance the performance of the management system by optimizing the sorting and data storage mechanisms for a larger amount of data.

7. Reference

Depth first search or DFS for a graph. (2023, June 9).

GeeksforGeeks. <https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/>

Heap queue (or heapq) in Python. (2023, March 22).

GeeksforGeeks. <https://www.geeksforgeeks.org/heap-queue-or-heapq-in-python/>

Implementation of hash table in Python using separate chaining. (2023, March 19).

GeeksforGeeks. <https://www.geeksforgeeks.org/implementation-of-hash-table-in-python-using-separate-chaining/>

Python linked list. (2023, July 12). GeeksforGeeks. <https://www.geeksforgeeks.org/python-linked-list/>

Python program for heap sort. (2024, January 10).

GeeksforGeeks. <https://www.geeksforgeeks.org/python-program-for-heap-sort/>

Python program for QuickSort. (2023, August 28).

GeeksforGeeks. <https://www.geeksforgeeks.org/python-program-for-quicksort/>